



Sessions 20 and 21

Public-Key Cryptography

[Session 20: video recording link](#)

[Session 21: video recording link](#)

- **Public-key cryptography** (also called asymmetric or two-key cryptography) uses two distinct keys:



Public Key:

- Can be shared openly
- Used to encrypt messages

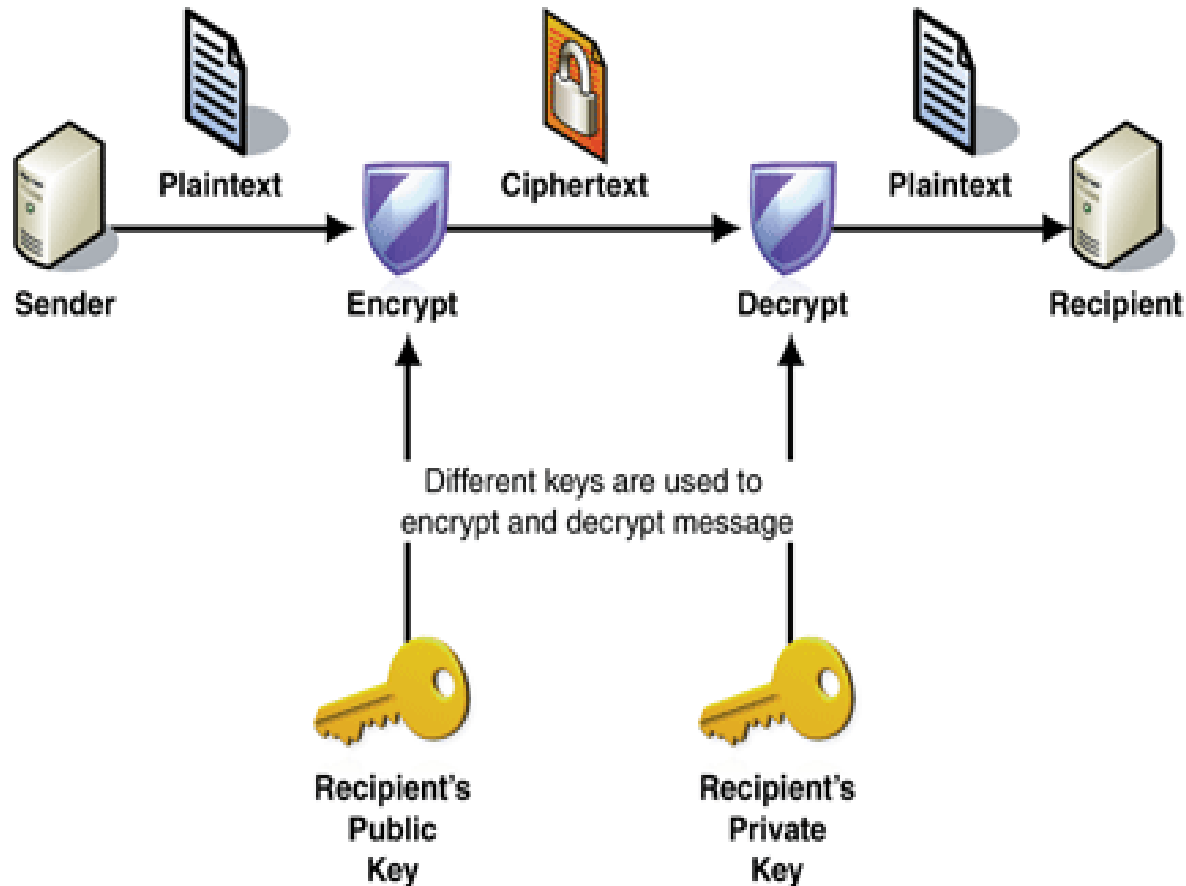


Private Key:

- Kept secret by the recipient
- Used to decrypt messages

- Asymmetric means:
 - The key used to encrypt cannot be used to decrypt
 - Only the private key can decrypt what the public key encrypts

Public-key Cryptography





Why Public-Key Cryptography?

- Designed to solve two major challenges:
- Key Distribution
 - Enables secure communication without relying on a trusted Key Distribution Center (KDC)
- Digital Signatures
 - Allows verification that a message is authentic and unaltered from the claimed sender
- History & Origin
 - Publicly introduced by Whitfield Diffie & Martin Hellman at Stanford University in 1976
 - Concept was previously known within classified/government communities

Public-Key Characteristics

- Public-key algorithms use two mathematically related keys with the following characteristics:
 - It is **computationally infeasible** to determine the private (decryption) key from the public (encryption) key and algorithm
 - **Encryption and decryption are efficient** when the correct key is used
 - In some schemes, **either key** can be used for encryption, with the other used for decryption

Public-Key Cryptography: Applications

- Public-key cryptography is used in a variety of important security applications:
 - Encryption/Decryption
 - Ensures confidentiality by encrypting data with a recipient's public key and decrypting with their private key
 - Digital Signatures
 - Provides message authenticity and integrity by signing with a private key and verifying with a public key
 - Key Exchange
 - Establishes a shared secret key between parties over an insecure channel (e.g., using Diffie–Hellman or RSA-based key exchange)
 - Authentication
 - Verifies the identity of users, devices, or systems (e.g., via certificate-based authentication)

Public-Key Cryptography: Security

- Like **symmetric key** systems, **brute-force** (exhaustive search) attacks are theoretically possible
- However, **key sizes are very large** (typically ≥ 1024 bits), making brute-force attacks impractical
- Security is based on a **large gap in difficulty** between:
 - Easy operations (e.g., encryption/decryption)
 - Hard problems (e.g., cryptanalysis)
- The **underlying hard** problem (e.g., factoring large numbers or discrete logarithms) is known, but **computationally infeasible** to solve in practice
- These schemes require the use of **very large numbers**, increasing computation time
- As a result, public-key schemes are generally **slower** than **symmetric key** schemes

RSA: Rivest–Shamir–Adleman (1977, MIT)

- Developed by **Ron Rivest**, **Adi Shamir**, and **Leonard Adleman**
- One of the most widely used **public-key cryptographic** systems
- Based on **modular exponentiation** in a **finite field of integers modulo a prime**
 - Exponentiation is computationally efficient:
 - Takes about $O((\log n)^3)$ operations
- Uses large integers (typically 1024 bits or more)
- Security relies on the difficulty of factoring large composite numbers
 - Integer factorization is computationally hard:
 - Takes roughly $O(e^{\sqrt{\log n \log \log n}})$ operations

1. Select two large prime numbers:

p and q

2. Compute the modulus:

$$n = p \times q$$

3. Calculate Euler's totient:

$$\varphi(n) = (p - 1)(q - 1)$$

4. Choose a public encryption exponent e :

Randomly selected such that:

$$1 < e < \varphi(n)$$

$$\gcd(e, \varphi(n)) = 1$$

5. Compute the private decryption exponent d

Solve:

$$e \cdot d \equiv 1 \pmod{\varphi(n)}, \text{ and} \\ 0 \leq d \leq n$$

6. Publish the public key:

$$KU = \{e, n\}$$

7. Keep the private key secret:

$$KR = \{d, p, q\}$$

RSA Use

- To encrypt a message M the sender:
 - obtains **public key** of recipient $K_U = \{e, n\}$
 - computes: $C = M^e \bmod n$, where $0 \leq M < n$
- To decrypt the ciphertext C the owner:
 - uses their private key $K_R = \{d, p, q\}$
 - computes: $M = C^d \bmod n$
- Note that the message M must be smaller than the modulus n (block if needed)

Why RSA Works

- Euler's Theorem:
- $a^{\phi(n)} \bmod n = 1$
 - where $\gcd(a, n) = 1$
- in RSA have:
 - $n = p \cdot q$
 - $\phi(n) = (p-1)(q-1)$
 - carefully chosen e & d to be inverses $\bmod \phi(n)$
 - hence $e \cdot d = 1 + k \cdot \phi(n)$ for some k
- hence :
$$C^d = (M^e)^d = M^{1+k \cdot \phi(n)} = M^1 \cdot (M^{\phi(n)})^k = M^1 \cdot (1)^k = M^1 = M \bmod n$$

RSA Example

1. Select primes: $p=17$ & $q=11$
2. Compute $n = pq = 17 \times 11 = 187$
3. Compute $\phi(n) = (p-1)(q-1) = 16 \times 10 = 160$
4. Select e : $\gcd(e, 160) = 1$; choose $e=7$
5. Determine d : $de=1 \pmod{160}$ and $d < 160$
Value is $d=23$ since $23 \times 7 = 161 = 10 \times 160 + 1$
6. Publish public key $KU = \{7, 187\}$
7. Keep secret private key $KR = \{23, 17, 11\}$

RSA Example cont

- Sample RSA encryption/decryption is:
- given message $M = 88$ ($88 < 187$)

- encryption:

$$C = 88^7 \bmod 187 = 11$$

- decryption:

$$M = 11^{23} \bmod 187 = 88$$

Exponentiation

- Can use the Square and Multiply Algorithm
- A fast, efficient algorithm for exponentiation
- Concept is based on repeatedly squaring base
- and multiplying in the ones that are needed to compute the result
- Look at binary representation of exponent
- Only takes $O(\log_2 n)$ multiples for number n
 - eg. $7^5 = 7^4 \cdot 7^1 = 3 \cdot 7 = 10 \pmod{11}$
 - eg. $3^{129} = 3^{128} \cdot 3^1 = 5 \cdot 3 = 4 \pmod{11}$

Exponentiation Algorithm

General Steps:

To compute $a^b \bmod n$:

1. Convert `b` to binary.
2. Initialize `result = 1`.
3. For each bit from **left to right** (or sometimes right to left):
 - Square the result.
 - If the bit is 1, multiply the result by `a`.
 - Always take mod `n` at each step to keep numbers small.

Exponentiation: Example

$$3^{129} \bmod 11$$

Step	Bit	Action	Calculation	Result (mod 11)
0	-	Initial value	-	1
1	1	Square + Multiply	$1^2 = 1, 1 \times 3 = 3$	3
2	0	Square only	$3^2 = 9$	9
3	0	Square only	$9^2 = 81 \bmod 11 = 4$	4
4	0	Square only	$4^2 = 16 \bmod 11 = 5$	5
5	0	Square only	$5^2 = 25 \bmod 11 = 3$	3
6	0	Square only	$3^2 = 9$	9
7	0	Square only	$9^2 = 81 \bmod 11 = 4$	4
8	1	Square + Multiply	$4^2 = 16, 16 \times 3 = 48$	$48 \bmod 11 = 4$

RSA Security

- Three approaches to attacking RSA:
 - Brute force key search (infeasible given size of numbers)
 - Mathematical attacks (based on difficulty of computing $\phi(n)$, by factoring modulus N)
 - Timing attacks (on running of decryption)

Factoring Problem

- Factor $n = p \cdot q$, hence find $\phi(n)$ and then d

Number of Decimal Digits	Number of Bits	Date Achieved
100	332	April 1991
110	365	April 1992
120	398	June 1993
129	428	April 1994
130	431	April 1996
140	465	February 1999
155	512	August 1999
160	530	April 2003
174	576	December 2003
200	663	May 2005
193	640	November 2005
232	768	December 2009
212	704	July 2012
240	795	December 2019
250	829	February 2020

Timing Attacks

- Developed in mid-1990's
- Exploit timing variations in operations
 - eg. multiplying by small vs large number
 - or IF's varying which instructions executed
- Infer operand size based on time taken
- RSA exploits time taken in exponentiation
- Countermeasures
 - Use constant exponentiation time
 - Add random delays
 - Blind values used in calculations

The Diffie-Hellman Algorithm

Introduction

- Discovered by Whitfield Diffie and Martin Hellman
 - “New Directions in Cryptography”
- Diffie-Hellman key agreement protocol
 - Exponential key agreement
 - Allows two users to exchange a secret key
 - Requires no prior secrets
 - Real-time over an untrusted network

Introduction

- Based on the difficulty of computing discrete logarithms of large numbers.
- No known successful attack strategies*
- Requires two large numbers, one prime (P), and (G), a primitive root of P

Discrete Logarithm problem

In **modular arithmetic**, a discrete logarithm is the inverse of exponentiation.

Given:

$$g^x \equiv y \pmod{p}$$

Here:

- g is the **base** (called the generator),
- p is a **prime modulus**,
- y is the **result**,
- the discrete logarithm problem is to **find** x .

Implementation

- p and g are both publicly available numbers
 - p is at least 512 bits
- Users pick private values a and b
- Compute public values
 - $x = g^a \bmod p$
 - $y = g^b \bmod p$
- Public values x and y are exchanged

Implementation

- Compute shared, private key
 - $k_a = y^a \bmod p$
 - $k_b = x^b \bmod p$
- Algebraically it can be shown that $k_a = k_b$
 - Users now have a symmetric secret key to encrypt

Example

- Alice and Bob get public numbers
 - $p = 23, g = 9$
- Alice and Bob compute public values
 - $X = 9^4 \bmod 23 = 6561 \bmod 23 = 6$
 - $Y = 9^3 \bmod 23 = 729 \bmod 23 = 16$
- Alice and Bob exchange public numbers

Example

- Alice and Bob compute symmetric keys
 - $k_a = y^a \bmod p = 16^4 \bmod 23 = 9$
 - $k_b = x^b \bmod p = 6^3 \bmod 23 = 9$
- Alice and Bob now can talk securely!

Applications

- Diffie-Hellman is currently used in many protocols, namely:
 - Secure Sockets Layer (SSL)/Transport Layer Security (TLS)
 - Secure Shell (SSH)
 - Internet Protocol Security (IPSec)
 - Public Key Infrastructure (PKI)